

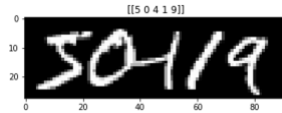
Practical work 3:

Embedded training of continuous Neuro-HMM

training without symbol segmentation

In this session we focus on moving from a pure HMM to a neuro_HMM. We focus on images of strings of digits that are of size $28 \times L$ as shown on figure below. $L < 140$ is the image width in pixels and may vary from one image to another because the digits may overlap, which makes the segmentation almost impossible without recognition.

The ground truth is the sequence of classes and does not provide any information about the position of the symbols in the image. Thus, training will proceed without segmentation with embedded training. Contrary to previous experiments, we will now consider the pixel columns of the image as the input frames of the sequences, we give up with discrete HMM.



Example of a 5 digits sequence generated from the MNIST dataset.

Neuro HMM architecture

A neuro HMM is a HMM where the emission probabilities are modeled with a neural network. A Multi-Layer Perceptron (MLP) is chosen, with a softmax activation function at its output, which estimates the state posterior probabilities:

$$\gamma_t(s_k) = P(q_t = s_k | O)$$

Every character Neuro_HMM will share the same Neural Network, which all in all makes a number of classes for the MLP equal to:

$$N_classes = n_symbols \times n_states$$

with $n_symbols = 10$ and $n_states = 5, \dots, 10, \dots, 15$

The Neuro-HMM equation is the following, where we see the state posteriors divided by the state priors to provide a normalized likelihood of the observation (in red). The transition probabilities and Initial and final probabilities (in blue) remain similar to those of a HMM

$$P(O, Q | M) \propto \pi_{q_1} \prod_t \frac{P(q_t | o_t)}{P(q_t)} \times P(q_t | q_{t-1})$$

This equation can simplify to the following one if we consider equi-probable state priors:

$$P(O, Q|M) \propto \pi_{q_1} \prod_t P(q_t|o_t) \times P(q_t|q_{t-1})$$

Embedded training with a Neuro HMM

We proceed using the following algorithm where one can see the use of **mini-batch** for training the neuro-hmm.

Also notice that this training scheme **only trains the Neural Network**, whereas the transition probabilities remain unchanged during training. The Markov Model remains unchanged with a uniform distribution for the left-right model (see the initialization procedure of the Neuro-HMM)

```
Models : the liste of digit models (the ten digit models)
NN = the initialized MLP
for iteration = 1,...,Max_EM_iter do
    for each mini_batch do
        EXPECTATION STEP
        for n = 1,...,mini_batch size do
            GT_Model = Create HMM for GT of example n°n
            Compute gamma probabilities for this example with GT_Model
            Derive the gamma probabilities for the 10 digit models
            Concatenate every frames of the batch
            Concatenate every frame predictions of the batch
        MAXIMISATION STEP
        Train the MLP to predict the Neuro-HMM predictions

    Compute train_LogLikelihood
    Compute valid_LogLikelihood

    if valid_LogLikelihood > previous_valid_LogLikelihood then
        Store the NN parameters
```

The **Embedded_Training()** method in **neuro_hmm.py** implements this algorithm.

Question 1 : explain the following lines of code

```
# shuffle the training data at each iteration
l=list(zip(train_data,gt_train))
random.shuffle(l)
train_data, gt_train = list(zip(*l))
```

Question 2 : in the **Embedded_Training()** method, what does the M step?

Question 3 : explain the **pred_on_batch()** method

Question 4 : explain the **GT_statesToSymbol_states()** method

Question 5: in the **forward()** and **backward()** code what are the **log_B** variables ? What is it for ?

Question 6 : Explain the **gamma()** method

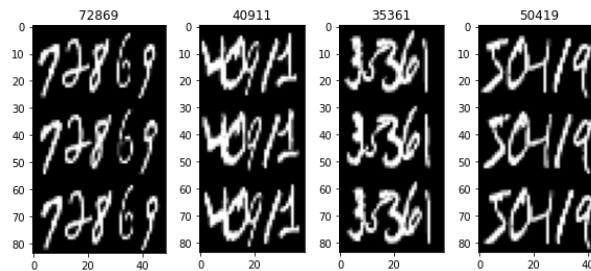
Question 7 : in the method **pred_on_batch()** explain the following lines of code ?

```
sum_prob = np.sum(B,axis=1)
for t in range(T):
    B[t,:] /=sum_prob[t]
```

Sliding window

In order to allow the MLP to take account of more context, we introduce a sliding window of size w_width , and stride w_stride . The sliding window takes into account every pixels in the window and is cast into a feature vector of size $28 * w_width$ pixels or features. This sliding window moves left to right over the image with a step of size w_stride . An image of width T is transformed into a sequence of length T/w_stride of feature vectors of size $28 * w_width$.

Some examples with $w_width = 3$ and $w_stride = 2$, providing feature vectors of size 84.



Question 8 : Train a model over 20 epochs with the following configuration:

N_epochs	N_states/symbol	#train seq	batch_size	w_width	W_stride
10	5	2000	128	3	2

Observe the training loss of the MLP and the Log Likelihood of the Neuro_HMM over time. Explain what happen?

Question 9: Evaluate the performance of the Neuro_HMM on the test set. Explain the recognition algorithm in detail?

What is the algorithm used, why?

What is the kind of HMM used during decoding, why?

Question 10: To provide a better initial model with missing data, we chose to process the first E step using a uniform distribution to predict the HMM states in the `pred_on_batch()` method. Observe the role of such initialization by viewing the training loss and compare this initialization with a pure random initialization.

Question 11 : Explore training and testing some other models with more and fewer states, and other parameters of the sliding window

Question 12 : Conclusion

- Discuss the capacity of Embedded Training to train with unknown segmentation.
- Discuss the capacity of Neuro_HMM with sliding window to model long range dependencies.
- What is the best configuration you would recommend, why?